

lua-xlsx - English documentation

Julien Freyermuth

Contents

1	lua-xlsx - English documentation	2
1.1	Contents	2
1.2	Architecture and Babet integration	2
1.3	Conventions	3
1.4	Writing XLSX files	3
1.4.1	xlsx.new([opts]) -> workbook	3
1.4.2	workbook:add_sheet([name]) -> sheet	3
1.4.3	sheet:write(row, col, value [, style]) -> sheet	4
1.4.4	sheet:append_row(values [, style])	4
1.4.5	sheet:write_rows(matrix [, style])	4
1.5	Styles, structure, layout, hyperlinks and formulas	4
1.5.1	xlsx.VERSION	4
1.5.2	xlsx.style([opts]) -> style	4
1.5.3	xlsx.is_style(value) and xlsx.style_options(style)	6
1.5.4	sheet:set_style(row, col, style) -> sheet	6
1.5.5	Column widths and row heights	6
1.5.6	Frozen panes	6
1.5.7	Automatic filters	6
1.5.8	Merged cells	6
1.5.9	Hyperlinks	6
1.5.10	Formulas	7
1.5.11	Data validation	7
1.5.12	Conditional formatting	8
1.5.13	Cell comments	8
1.5.14	Hidden rows and columns	9
1.5.15	Sheet properties and active sheet	9
1.5.16	Defined names	9
1.5.17	Complete 1.3.0 example (features retained)	9
1.5.18	workbook:save(path [, opts])	10
1.5.19	xlsx.write_rows(path, matrix [, opts])	11
1.6	Reports, charts, images, and printing	11
1.6.1	PNG and JPEG images	11
1.6.2	Simple charts	12
1.6.3	Structured Excel tables	12
1.6.4	Page setup	13
1.6.5	Page margins	13
1.6.6	Headers and footers	13
1.6.7	Print area and repeated titles	13
1.6.8	Reading 1.4 metadata	14
1.6.9	Complete 1.4.0 report example	14
1.7	Finishing, advanced visualization, and protection	14
1.7.1	Rich text inside a cell	15
1.7.2	Visual conditional formatting	15
1.7.3	Sparklines	17
1.7.4	Advanced charts	18
1.7.5	Cell protection	20
1.7.6	Worksheet protection	20
1.7.7	Workbook structure protection	20

1.7.8	Manual page breaks	21
1.7.9	Document properties	21
1.7.10	Reading 1.5 information	21
1.7.11	Complete 1.5.0 example	22
1.8	1900 and 1904 dates	23
1.9	Reading XLSX files	23
1.9.1	<code>xlsx.open(path [, opts])</code>	23
1.9.2	Workbook methods	23
1.9.3	Values and formulas	24
1.9.4	Styles	24
1.9.5	Layout	24
1.9.6	Data validation, conditional formatting and comments	24
1.9.7	Merged cells	25
1.9.8	Hyperlinks	25
1.9.9	Dimensions and row iteration	25
1.10	ZIP validation and limits	25
1.11	DataFrame	26
1.11.1	Construction	26
1.11.2	Introspection	26
1.11.3	Non-destructive transformations	26
1.11.4	Grouping and aggregation	27
1.11.5	Output	27
1.12	Error contract	27
1.13	Tests and interoperability	27
1.14	Building the PDFs	28
1.15	Known limitations	28

1 lua-xlsx - English documentation

Reference for the `xlsx` and `dataframe` modules. Babet with Lua 5.5 is the primary target, while the pure-Lua core remains compatible with standard Lua 5.3+.

Documentation for lua-xlsx 1.5.0.

1.1 Contents

- [Architecture and Babet integration](#)
- [Conventions](#)
- [Writing XLSX files](#)
- [Styles, structure, layout, hyperlinks and formulas](#)
- [Reports, charts, images, and printing](#)
- [Finishing, advanced visualization, and protection](#)
- [1900 and 1904 dates](#)
- [Reading XLSX files](#)
- [ZIP validation and limits](#)
- [DataFrame](#)
- [Error contract](#)
- [Tests and interoperability](#)
- [Building the PDFs](#)
- [Known limitations](#)

1.2 Architecture and Babet integration

`xlsx.lua` and `dataframe.lua` require no external Lua module. When the global `babet` table is available, `xlsx.lua` automatically uses:

Babet function	lua-xlsx use
<code>babet.writeFileAtomic</code>	atomic and durable XLSX publication
<code>babet.archive.test</code>	full ZIP validation before parsing
<code>babet.fileSize</code>	file-size check before loading into memory
<code>babet.crc32</code>	native CRC-32 calculation

If one function is absent, the pure-Lua path is retained for that operation.

To disable Babet for one call:

```
assert(wb:save("report.xlsx", { use_babet = false }))
```

```
local read, err = xlsx.open("report.xlsx", {
    use_babet = false,
})
assert(read, err)
```

This is mostly useful for testing the fallback. Production code running in Babet should normally keep the default true.

1.3 Conventions

- `sheet:write(row, col, value [, style])` and `sheet:read(row, col)` use **0-based** indices. Cell A1 is (0, 0).
- `sheet:rows()` yields **1-based** Lua arrays. `row[1]` is column A and `row.n` is the width.
- An empty cell is `nil`. Boolean `false` never collapses to `nil`.
- Dates are returned as ISO 8601 strings.
- XLSX bounds are enforced: rows 0..1048575, columns 0..16383.
- Written strings must be valid UTF-8 and contain only XML 1.0 characters.
- DataFrame transformations return newly copied row records.

1.4 Writing XLSX files

1.4.1 `xlsx.new([opts]) -> workbook`

Option:

Option	Default	Values
<code>date_system</code>	"1900"	"1900" or "1904"

```
local wb1900 = xlsx.new()
local wb1904 = xlsx.new({ date_system = "1904" })
```

Unknown options are rejected.

1.4.2 `workbook:add_sheet([name]) -> sheet`

The default name is SheetN.

Rules:

- 1 to 31 **Unicode characters**, not bytes;
- valid UTF-8 and XML 1.0;
- none of `:`, `\`, `/`, `?`, `*`, `[` or `]`;
- no leading or trailing apostrophe;
- no exact duplicate or ASCII case-only duplicate.

```
local wb = xlsx.new()
local data = wb:add_sheet("Data")
local unicode = wb:add_sheet(string.rep("é", 20))
```

1.4.3 sheet:write(row, col, value [, style]) -> sheet

Accepted values: string, finite number, boolean, date value, formula value or nil. The optional style must be created by xlsx.style.

```
local sh = xlsx.new():add_sheet("Example")
sh:write(0, 0, "Text")
sh:write(0, 1, 42)
sh:write(0, 2, 3.14)
sh:write(0, 3, false)
sh:write(0, 4, xlsx.date(2026, 7, 18))
sh:write(0, 5, xlsx.formula("SUM(B1:C1)"))
```

NaN, infinities, unsupported types and out-of-range indices raise a Lua error.

1.4.4 sheet:append_row(values [, style])

```
sh:append_row({ "Alice", 30, true })
sh:append_row({ "Bob", 25, false })
```

1.4.5 sheet:write_rows(matrix [, style])

```
sh:write_rows({
  { "Name", "Score" },
  { "Alice", 18 },
  { "Bob", 15 },
})
```

1.5 Styles, structure, layout, hyperlinks and formulas

Version 1.5.0 writes these elements and also exposes the supported subset through the reading API. style, formula, hyperlink, and rich_text objects are immutable.

1.5.1 xlsx.VERSION

```
assert(xlsx.VERSION == "1.5.0")
```

1.5.2 xlsx.style([opts]) -> style

A style must be created with xlsx.style; raw option tables are rejected by write() and set_style().

Option	Type	Values
bold	boolean	bold font
italic	boolean	italic font
underline	string	none, single, double
strike	boolean	strikethrough
font_name	string	non-empty UTF-8 name, up to 255 bytes
font_size	number	1 to 409 points
font_color	string	RGB RRGGBB, #RRGGBB, or ARGB AARRGGBB
fill_color	string	solid RGB or ARGB fill
horizontal	string	left, center, right, justify
vertical	string	top, center, bottom, justify
wrap_text	boolean	wrap text
number_format	string	alias or Excel format code
locked	boolean	locked when worksheet protection is enabled

Option	Type	Values
hidden	boolean	formula hidden when worksheet protection is enabled
border	table	left, right, top, bottom sides

Six-digit RGB colors automatically receive an opaque FF alpha channel.

Bold and italic:

```
local emphasis = xlsx.style({ bold = true, italic = true })
sh:write(0, 0, "Important", emphasis)
```

Font name and size:

```
local title = xlsx.style({
  font_name = "Liberation Sans",
  font_size = 16,
})
sh:write(1, 0, "Title", title)
```

Underlining and strikethrough:

```
sh:write(2, 0, "Single", xlsx.style({ underline = "single" }))
sh:write(3, 0, "Double", xlsx.style({ underline = "double" }))
sh:write(4, 0, "Old value", xlsx.style({ strike = true }))
```

Font and fill colors:

```
local colored = xlsx.style({
  font_color = "FFFFFF",
  fill_color = "4472C4",
})
```

Alignment and wrapping:

```
local wrapped = xlsx.style({
  horizontal = "center",
  vertical = "center",
  wrap_text = true,
})
```

Number-format aliases:

Alias	Generated Excel code
general	no custom format
integer	0
decimal	0.00
percent	0.00%
currency_eur	#,##0.00 "€"
currency_usd	\$#,##0.00
date	yyyy-mm-dd
datetime	yyyy-mm-dd hh:mm:ss

```
sh:write(0, 1, 42, xlsx.style({ number_format = "integer" }))
sh:write(1, 1, 3.14159, xlsx.style({ number_format = "decimal" }))
sh:write(2, 1, 0.125, xlsx.style({ number_format = "percent" }))
sh:write(3, 1, 19.90, xlsx.style({ number_format = "currency_eur" }))
```

Custom code:

```
sh:write(4, 1, 12.3456, xlsx.style({ number_format = '0.000 "kg"' }))
```

Date values automatically receive a date format if the style does not specify one.

1.5.2.1 Borders Each side is a table with style and optional color. Supported styles are thin, medium, thick, dashed, dotted, and double.

```
local framed = xlsx.style({
  border = {
    left = { style = "thin", color = "808080" },
    right = { style = "thin", color = "808080" },
    top = { style = "double", color = "000000" },
    bottom = { style = "double", color = "000000" },
  },
})
```

1.5.3 xlsx.is_style(value) and xlsx.style_options(style)

```
assert(xlsx.is_style(framed))
local opts = xlsx.style_options(framed)
style_options() returns a deep copy.
```

1.5.4 sheet:set_style(row, col, style) -> sheet

```
sh:set_style(0, 0, xlsx.style({ bold = true }))
sh:set_style(1, 0, xlsx.style({ fill_color = "E2F0D9" }))
sh:set_style(0, 0, nil)
```

1.5.5 Column widths and row heights

```
sh:set_column_width(0, 24) -- 0.1 to 255
sh:set_row_height(0, 28) -- 0.1 to 409.5 points
```

1.5.6 Frozen panes

```
sh:freeze_panes(1, 0)
sh:freeze_panes(0, 1)
sh:freeze_panes(1, 1)
sh:freeze_panes(0, 0)
```

1.5.7 Automatic filters

```
sh:set_auto_filter()
sh:set_auto_filter("A1:D100")
sh:set_auto_filter(false)
```

1.5.8 Merged cells

```
sh:write(0, 0, "Report")
sh:merge_cells("A1:D1")
sh:merge_cells(2, 0, 2, 3) -- A3:D3, zero-based coordinates
sh:unmerge_cells("A1:D1")
```

Single-cell, reversed, and overlapping merges are rejected. Only the top-left cell keeps its value, style, and hyperlink.

1.5.9 Hyperlinks

External hyperlink value:

```
sh:write(0, 0, xlsx.hyperlink(
  "https://example.com/?a=1&b=2",
  "Open website",
  { tooltip = "External website" }
))
```

Internal hyperlink:

```
sh:write(1, 0, xlsx.hyperlink(
  "'Details'!A1",
  "View details",
  { internal = true }
))
```

Apply or remove a hyperlink on an existing cell:

```
sh:write(2, 0, "Documentation")
sh:set_hyperlink(2, 0, "https://example.com/docs")
sh:set_hyperlink(3, 0, "'Summary'!B2", { internal = true })
sh:remove_hyperlink(2, 0)
```

1.5.10 Formulas

```
sh:write(3, 1, xlsx.formula("SUM(B1:B3)"))
sh:write(3, 2, xlsx.formula("=AVERAGE(C1:C3)"))
sh:write(4, 1, xlsx.formula("SUM(B2:B4)", 42.5))
```

The optional cached value may be a string, number, or boolean. It is not verified against the formula. lua-xlsx does not calculate formulas.

1.5.11 Data validation

1.5.11.1 sheet:add_data_validation(range, opts) -> sheet Supported types are list, whole, decimal, date, time, text_length and custom. Numeric-like types accept between, not_between, equal, not_equal, greater_than, less_than, greater_or_equal and less_or_equal.

Inline list:

```
sheet:add_data_validation("A2:A100", {
  type = "list",
  values = { "Yes", "No", "Pending" },
})
```

Inline choices cannot contain commas or double quotes and the generated formula is limited to 255 bytes. Use a defined name for larger lists:

```
workbook:define_name("Statuses", "'Settings'!$A$1:$A$20")
sheet:add_data_validation("A2:A100", {
  type = "list",
  formula = "Statuses",
})
```

Whole number between two bounds:

```
sheet:add_data_validation("B2:B100", {
  type = "whole",
  operator = "between",
  minimum = 0,
  maximum = 100,
})
```

Date and custom formula:

```
sheet:add_data_validation("C2:C100", {
  type = "date",
  operator = "greater_than",
  value = xlsx.date(2026, 1, 1),
})
```

```
sheet:add_data_validation("D2:D100", {
  type = "custom",
```

```

    formula = "MOD(D2,2)=0",
})

```

Input and error messages use `allow_blank`, `show_dropdown`, `show_input_message`, `prompt_title`, `prompt`, `show_error_message`, `error_style`, `error_title` and `error`. `error_style` accepts `stop`, `warning` or `information`.

`remove_data_validation(range)` removes rules attached exactly to that range. `get_data_validations()` returns independent copies.

1.5.12 Conditional formatting

1.5.12.1 `sheet:add_conditional_format(range, opts) -> sheet` A style created by `xlsx.style()` is required. Conditional styles support font, solid fill and the four borders. Number formats and alignment are deliberately rejected.

```

local negative = xlsx.style({
    font_color = "9C0006",
    fill_color = "FFC7CE",
})

```

```

sheet:add_conditional_format("B2:B100", {
    type = "cell",
    operator = "less_than",
    value = 0,
    style = negative,
})

```

Other supported rule types:

```

sheet:add_conditional_format("C2:C100", {
    type = "contains_text",
    text = "urgent",
    style = xlsx.style({ bold = true, font_color = "C00000" }),
})

```

```

sheet:add_conditional_format("D2:D100", {
    type = "blanks", -- also not blanks or duplicate
    style = xlsx.style({ fill_color = "FFFF00" }),
})

```

```

sheet:add_conditional_format("A2:D100", {
    type = "custom",
    formula = "$D2>1000",
    stop_if_true = true,
    style = xlsx.style({ bold = true, fill_color = "C6EFCE" }),
})

```

Cell rules use the same comparison operators as validation and accept either value or minimum/maximum. `remove_conditional_format(range)` removes rules attached exactly to the range. Priorities follow insertion order.

1.5.13 Cell comments

```

sheet:set_comment(2, 0, {
    author = "Julien",
    text = "Check this value before publishing.",
})

```

```

sheet:remove_comment(2, 0)

```

Comments are classic Excel notes with plain text. Rich text, custom dimensions and custom positions are not exposed.

1.5.14 Hidden rows and columns

```
sheet:set_row_hidden(4, true)
sheet:set_column_hidden(7, true)
```

Pass false to make the row or column visible again. Indices are zero-based.

1.5.15 Sheet properties and active sheet

```
sheet:set_tab_color("4472C4")
sheet:set_visibility("hidden") -- visible, hidden or very_hidden
workbook:set_active_sheet("Summary")
```

At least one sheet must remain visible and the active sheet must be visible. `workbook:get_active_sheet()` returns the writable sheet object.

1.5.16 Defined names

```
workbook:define_name("VATRate", "'Settings'!$B$2")
workbook:define_name("Products", "'Data'!$A$2:$D$100")
workbook:define_name("LocalTotal", "'Summary'!$B$10", {
    local_sheet = "Summary",
    hidden = true,
    comment = "Technical local name",
})
```

Names use a conservative ASCII syntax, are case-insensitively unique within a scope, and cannot look like a cell reference.

```
for _, item in ipairs(workbook:get_defined_names()) do
    print(item.name, item.reference, item.local_sheet)
end
```

```
workbook:remove_defined_name("VATRate")
```

1.5.17 Complete 1.3.0 example (features retained)

```
local xlsx = require("xlsx")
local wb = xlsx.new()
local sh = wb:add_sheet("Sales")

local title = xlsx.style({
    bold = true,
    underline = "double",
    font_name = "Liberation Sans",
    font_size = 16,
    font_color = "FFFFFF",
    fill_color = "4472C4",
    horizontal = "center",
    border = { bottom = { style = "double", color = "1F4E78" } },
})
local header = xlsx.style({
    bold = true,
    fill_color = "D9EAF7",
    horizontal = "center",
    border = {
        left = { style = "thin", color = "808080" },
        right = { style = "thin", color = "808080" },
        top = { style = "thin", color = "808080" },
        bottom = { style = "thin", color = "808080" },
    },
})
local money = xlsx.style({ number_format = "currency_eur" })
```

```

sh:write(0, 0, "Sales report", title)
sh:merge_cells("A1:E1")
sh:append_row({ "Product", "Price", "Qty", "Total", "Link" }, header)
sh:append_row({ "Keyboard", 39.90, 2 })
sh:append_row({ "Mouse", 24.50, 3 })
sh:write(2, 1, 39.90, money)
sh:write(3, 1, 24.50, money)
sh:write(2, 3, xlsx.formula("B3*C3", 79.80), money)
sh:write(3, 3, xlsx.formula("B4*C4", 73.50), money)
sh:write(2, 4, xlsx.hyperlink("https://example.com/keyboard", "Product"))
sh:set_column_width(0, 24)
sh:set_row_height(0, 30)
sh:freeze_panes(2, 1)
sh:set_auto_filter("A2:E4")
assert(wb:save("sales.xlsx"))

```

1.5.18 workbook:save(path [, opts])

Option	Default	Effect
overwrite	true	replace an existing file
durable	true	request durable synchronization in Babet
permissions	0644	final Babet file mode
use_babet	true	use writeFileAtomic when available

Simple write:

```

local ok, err = wb:save("report.xlsx")
assert(ok, err)

```

Refuse overwrite:

```

local ok, err = wb:save("report.xlsx", {
    overwrite = false,
})

```

Rebuildable cache:

```

assert(wb:save("cache.xlsx", {
    durable = false,
}))

```

Private permissions:

```

assert(wb:save("private.xlsx", {
    permissions = tonumber("600", 8),
}))

```

Combined example:

```

local ok, err = wb:save("export/report.xlsx", {
    overwrite = true,
    durable = true,
    permissions = tonumber("640", 8),
    use_babet = true,
})
assert(ok, err)

```

With Babet, this calls `babet.writeFileAtomic`. Standard Lua writes a temporary file in the same directory, checks write, flush and close, then renames it. The fallback is atomic on ordinary POSIX filesystems but does not duplicate all of Babet's descriptor confinement and fsync guarantees.

1.5.19 `xlsx.write_rows(path, matrix [, opts])`

Options are `sheet`, `date_system`, `overwrite`, `durable`, `permissions` and `use_babet`.

```
assert(xlsx.write_rows("result.xlsx", {
  { "x", "y" },
  { 1, 2 },
  { 3, 4 },
}, {
  sheet = "Results",
  date_system = "1900",
  overwrite = true,
}))
```

1.6 Reports, charts, images, and printing

Version 1.4.0 adds the components needed for simple reports and dashboards. One worksheet drawing can contain several images and charts; structured tables and printing settings remain independent.

1.6.1 PNG and JPEG images

1.6.1.1 `sheet:add_image(path, row, col [, opts])`

```
sh:add_image("logo.png", 0, 5)
```

Options:

Option	Default	Meaning
width	native	width in pixels, 1 to 10000
height	native	height in pixels, 1 to 10000
alt_text	none	accessible description, up to 1000 bytes
name	generated	image object name, up to 255 bytes

Providing only one dimension preserves the aspect ratio:

```
sh:add_image("logo.png", 1, 5, { width = 160 })
sh:add_image("photo.jpg", 8, 0, { height = 120 })
sh:add_image("logo.png", 0, 5, {
  width = 160,
  alt_text = "lua-xlsx project logo",
  name = "Main logo",
})
```

Only PNG and JPEG are supported. The format is detected from the actual bytes.

1.6.1.2 `sheet:add_image_data(data, format, row, col [, opts])`

```
local file = assert(io.open("logo.png", "rb"))
local bytes = assert(file:read("a"))
assert(file:close())
sh:add_image_data(bytes, "png", 0, 5, { width = 160 })
```

`format` accepts `png`, `jpeg`, or `jpg`.

1.6.1.3 `sheet:remove_image(index)`

```
sh:remove_image(1)
```

1.6.2 Simple charts

1.6.2.1 sheet:add_chart(opts) Supported types are line, column, and bar.

```
sh:add_chart({
  type = "line",
  title = "Monthly sales",
  categories = "A2:A13",
  series = {
    { name_ref = "B1", values = "B2:B13" },
  },
  row = 1,
  col = 5,
})
```

Multiple series and explicit dimensions:

```
sh:add_chart({
  type = "column",
  categories = "A2:A13",
  series = {
    { name = "Actual", values = "B2:B13" },
    { name = "Target", values = "C2:C13" },
  },
  row = 15,
  col = 0,
  width = 700,
  height = 360,
  legend = true,
})
```

Option	Default
type	column
title	none
categories	required
series	required, 1 to 255
row, col	0, 0
width, height	640, 360
legend	true

lua-xlsx stores range formulas but does not calculate or cache chart points.

1.6.2.2 sheet:remove_chart(index)

```
sh:remove_chart(1)
```

1.6.3 Structured Excel tables

1.6.3.1 sheet:add_table(ref [, opts]) The range must include a header row and at least one data row. Headers must be non-empty unique strings.

```
sh:add_table("A1:D100", { name = "SalesTable" })
sh:add_table("A1:D100", {
  name = "SalesTable",
  style = "TableStyleMedium4",
  show_row_stripes = true,
  show_column_stripes = false,
})
```

Options: name, style, show_first_column, show_last_column, show_row_stripes, and show_column_stripes. Table names are workbook-wide unique and tables on one sheet may not overlap.

1.6.3.2 sheet:remove_table(name)

```
sh:remove_table("SalesTable")
```

1.6.4 Page setup

1.6.4.1 sheet:set_page_setup(opts)

```
sh:set_page_setup({ orientation = "landscape", paper_size = "a4" })
```

Fit to one page wide with unlimited height:

```
sh:set_page_setup({  
  orientation = "landscape",  
  paper_size = "a4",  
  fit_to_width = 1,  
  fit_to_height = 0,  
})
```

Fixed scale:

```
sh:set_page_setup({ scale = 85 })
```

scale accepts 10 to 400 and cannot be combined with fit settings. The method also accepts horizontal_centered, vertical_centered, grid_lines, and headings. Paper can be letter, legal, a3, a4, a5, or an integer OOXML paper identifier. Passing false clears the setup.

1.6.5 Page margins

1.6.5.1 sheet:set_page_margins(opts) Values are inches:

```
sh:set_page_margins({  
  left = 0.4, right = 0.4,  
  top = 0.5, bottom = 0.5,  
  header = 0.3, footer = 0.3,  
})
```

Missing fields use conventional Excel defaults. Passing false clears the explicit margins.

1.6.6 Headers and footers

1.6.6.1 sheet:set_header_footer(opts)

```
sh:set_header_footer({  
  header_left = "lua-xlsx",  
  header_center = "Annual report",  
  header_right = "&D",  
  footer_left = "Confidential",  
  footer_center = "&F",  
  footer_right = "Page &P / &N",  
})
```

Excel codes such as &P, &N, &D, and &F are preserved. different_first and different_odd_even are supported. Passing false clears the configuration.

1.6.7 Print area and repeated titles

1.6.7.1 sheet:set_print_area(ref)

```
sh:set_print_area("A1:J60")
```

1.6.7.2 sheet:set_print_titles(opts)

```
sh:set_print_titles({ rows = "1:1" })  
sh:set_print_titles({ rows = "1:2", columns = "A:A" })
```

Rows use 1:2; columns use A:B. Passing false removes the corresponding workbook-defined names.

1.6.8 Reading 1.4 metadata

```
local images = sh:get_images()
local charts = sh:get_charts()
local tables = sh:get_tables()
local margins = sh:get_page_margins()
local page = sh:get_page_setup()
local header_footer = sh:get_header_footer()
local print_area = sh:get_print_area()
local repeat_rows, repeat_cols = sh:get_print_titles()
```

Image metadata includes binary bytes, format, anchor, size, name, and alternative text. Chart metadata includes type, title, category/series range formulas, and anchor. Table metadata includes name, range, style, and headers.

1.6.9 Complete 1.4.0 report example

```
local xlsx = require("xlsx")
local wb = xlsx.new()
local sh = wb:add_sheet("Dashboard")
sh:write_rows({
  { "Month", "Actual", "Target" },
  { "January", 120, 100 },
  { "February", 135, 130 },
  { "March", 128, 140 },
})
sh:add_table("A1:C4", { name = "SalesTable", style = "TableStyleMedium4" })
sh:add_image("logo.png", 0, 5, { width = 140, alt_text = "Company logo" })
sh:add_chart({
  type = "column",
  title = "Actual versus target",
  categories = "A2:A4",
  series = {
    { name_ref = "B1", values = "B2:B4" },
    { name_ref = "C1", values = "C2:C4" },
  },
  row = 6, col = 0, width = 720, height = 380,
})
sh:set_page_setup({
  orientation = "landscape", paper_size = "a4",
  fit_to_width = 1, fit_to_height = 0,
  horizontal_centered = true,
})
sh:set_page_margins({ left = 0.4, right = 0.4, top = 0.5, bottom = 0.5 })
sh:set_header_footer({ header_center = "Sales dashboard", footer_right = "Page
↵ &P / &N" })
sh:set_print_area("A1:J30")
sh:set_print_titles({ rows = "1:1", columns = "A:A" })
assert(wb:save("dashboard.xlsx"))
```

1.7 Finishing, advanced visualization, and protection

Version 1.5.0 extends the reporting features introduced in 1.4.0 with richer visual elements, classic XLSX protection, document metadata, and rich text.

1.7.1 Rich text inside a cell

1.7.1.1 `xlsx.rich_text(runs)` -> `rich_text` Creates a value made of several text runs. Every run requires text and may set:

- bold;
- italic;
- underline: "single" or "double";
- strike;
- font_name;
- font_size;
- font_color.

```
local warning = xlsx.rich_text({
  { text = "Warning: ", bold = true, font_color = "FF0000" },
  { text = "the deadline is Friday.", italic = true },
})
```

```
sheet:write(0, 0, warning)
```

The convenience method avoids creating the value explicitly:

```
sheet:write_rich_text(1, 0, {
  { text = "Total: ", bold = true },
  { text = "€1,250", font_color = "008000", underline = "single" },
})
```

A complete cell style may be supplied separately:

```
sheet:write_rich_text(2, 0, {
  { text = "Status: ", bold = true },
  { text = "approved", italic = true, font_color = "008000" },
}, xlsx.style({ fill_color = "E2F0D9", wrap_text = true }))
```

1.7.1.2 `xlsx.is_rich_text(value)` and `xlsx.rich_text_runs(value)` `is_rich_text()` identifies a rich-text value. `rich_text_runs()` returns an independent copy of its runs.

```
local rich = sheet:get_rich_text(0, 0)
if rich and xlsx.is_rich_text(rich) then
  for _, run in ipairs(xlsx.rich_text_runs(rich)) do
    print(run.text, run.bold, run.italic, run.font_color)
  end
end
```

`sheet:read()` still returns the concatenated plain text, preserving a simple reading path for programs that do not need the individual runs.

1.7.2 Visual conditional formatting

The following rules do not use a differential style. They generate native spreadsheet visualizations.

1.7.2.1 Two-color scale

```
sheet:add_conditional_format("B2:B100", {
  type = "color_scale",
  min_color = "F8696B",
  max_color = "63BE7B",
})
```

The default thresholds are min and max.

1.7.2.2 Three-color scale

```
sheet.add_conditional_format("B2:B100", {
    type = "color_scale",
    min_type = "number",
    min_value = 0,
    min_color = "F8696B",
    mid_type = "percentile",
    mid_value = 50,
    mid_color = "FFEB84",
    max_type = "number",
    max_value = 100,
    max_color = "63BE7B",
})
```

Threshold types are min, max, number, percent, percentile, and formula. min and max thresholds do not accept a value.

1.7.2.3 Data bar

```
sheet.add_conditional_format("C2:C100", {
    type = "data_bar",
    start_type = "min",
    end_type = "max",
    color = "5B9BD5",
    show_value = true,
})
```

Numeric bounds can be used explicitly:

```
sheet.add_conditional_format("C2:C100", {
    type = "data_bar",
    start_type = "number",
    start_value = 0,
    end_type = "number",
    end_value = 500,
    color = "70AD47",
    show_value = false,
})
```

1.7.2.4 Icon set

```
sheet.add_conditional_format("D2:D100", {
    type = "icon_set",
    icons = "3_traffic_lights",
    value_type = "number",
    values = { 0, 50, 80 },
    show_value = true,
    reverse = false,
})
```

Supported sets:

- 3_arrows, 3_arrows_gray, 3_flags;
- 3_traffic_lights, 3_traffic_lights_rimmed;
- 3_signs, 3_symbols, 3_symbols_circled;
- 4_arrows, 4_arrows_gray, 4_ratings, 4_traffic_lights;
- 5_arrows, 5_arrows_gray, 5_ratings, 5_quarters.

The threshold count must match the number in the set name.

1.7.2.5 Top, bottom, and average rules


```

local highlight = xlsx.style({
  bold = true,
  font_color = "006100",
  fill_color = "C6EFCE",
})

sheet:add_conditional_format("B2:B100", {
  type = "top",
  rank = 10,
  percent = false,
  style = highlight,
})

sheet:add_conditional_format("C2:C100", {
  type = "bottom",
  rank = 5,
  percent = true,
  style = xlsx.style({ fill_color = "FFC7CE" }),
})

sheet:add_conditional_format("D2:D100", {
  type = "above_average",
  style = highlight,
})

sheet:add_conditional_format("E2:E100", {
  type = "below_average",
  style = xlsx.style({ font_color = "9C0006" }),
})

```

1.7.3 Sparklines

1.7.3.1 sheet:add_sparkline(target, source [, opts]) -> sheet Adds a mini-chart to one cell. The target must be a single cell and the source a range on the same worksheet.

Line sparkline:

```

sheet:add_sparkline("N2", "B2:M2", {
  type = "line",
  color = "4472C4",
  show_markers = true,
  show_high = true,
  show_low = true,
  high_color = "00B050",
  low_color = "C00000",
})

```

Column sparkline:

```

sheet:add_sparkline("N3", "B3:M3", {
  type = "column",
  color = "5B9BD5",
  show_negative = true,
  negative_color = "C00000",
  show_first = true,
  show_last = true,
})

```

Win/loss sparkline:

```

sheet:add_sparkline("N4", "B4:M4", {
  type = "win_loss",
  color = "70AD47",
})

```

```
right_to_left = false,
})
```

Available options are type, color, negative_color, high_color, low_color, first_color, last_color, show_markers, show_high, show_low, show_first, show_last, show_negative, and right_to_left.

```
sheet.remove_sparkline("N3")
local sparklines = sheet.get_sparklines()
```

Sparklines use the standard OOXML x14 extension. Excel and LibreOffice understand it. openpyxl 3.1.5 warns that it does not support that extension and removes it when it saves the workbook again.

1.7.4 Advanced charts

sheet.add_chart() now accepts:

- line;
- column;
- bar;
- pie;
- doughnut;
- area;
- scatter.

1.7.4.1 Pie chart

```
sheet.add_chart({
    type = "pie",
    title = "Sales distribution",
    categories = "A2:A6",
    series = {
        { name = "Sales", values = "B2:B6" },
    },
    legend_position = "bottom",
    data_labels = {
        show_percent = true,
        show_category = true,
        position = "best_fit",
    },
    row = 1,
    col = 4,
})
```

A pie or doughnut chart requires exactly one series.

1.7.4.2 Doughnut chart

```
sheet.add_chart({
    type = "doughnut",
    title = "Distribution",
    categories = "A2:A6",
    series = {
        { name_ref = "B1", values = "B2:B6" },
    },
    hole_size = 65,
    legend_position = "right",
    row = 18,
    col = 4,
})
```

hole_size accepts a value from 10 to 90.

1.7.4.3 Stacked area chart

```
sheet.add_chart({
  type = "area",
  title = "Cumulative trend",
  categories = "A2:A13",
  grouping = "stacked",
  series = {
    { name = "Product A", values = "B2:B13", color = "4472C4" },
    { name = "Product B", values = "C2:C13", color = "ED7D31" },
  },
  x_axis = { title = "Month" },
  y_axis = {
    title = "Sales",
    min = 0,
    max = 10000,
    number_format = '$#,##0',
    major_gridlines = true,
  },
  legend_position = "top",
  row = 35,
  col = 4,
})
```

grouping accepts standard, stacked, and percent_stacked for line, area, column, and bar charts.

1.7.4.4 Scatter chart

```
sheet.add_chart({
  type = "scatter",
  title = "Temperature and consumption",
  x_values = "A2:A50",
  series = {
    {
      name = "Consumption",
      values = "B2:B50",
      color = "70AD47",
      marker = "circle",
      line_width = 2,
      smooth = true,
    },
  },
  x_axis = { title = "Temperature", min = -10, max = 40 },
  y_axis = { title = "kWh", min = 0, major_gridlines = true },
  legend = false,
  row = 52,
  col = 4,
})
```

Each scatter series may provide its own x_values. Otherwise the chart-level x_values is used.

1.7.4.5 Shared options Legend positions: right, left, top, bottom, top_right.

Series options:

- name or name_ref;
- values;
- x_values for scatter charts;
- color;
- marker;
- line_width, from 0.25 to 20 points;
- smooth.

Markers: none, circle, dash, diamond, dot, picture, plus, square, star, triangle, x.

Data labels accept show_value, show_percent, show_category, show_series_name, and position. Positions: center, inside_end, inside_base, outside_end, best_fit, left, right, top, bottom.

Axes accept title, min, max, number_format, and major_gridlines. The minimum must be strictly lower than the maximum.

1.7.5 Cell protection

The locked and hidden options are part of `xlsx.style()`:

```
local input_cell = xlsx.style({
  fill_color = "FFF2CC",
  locked = false,
})
```

```
local protected_formula = xlsx.style({
  number_format = "0.00",
  locked = true,
  hidden = true,
})
```

```
sheet:write(1, 1, 12.5, input_cell)
sheet:write(1, 2, xlsx.formula("=B2*1.2", 15), protected_formula)
```

These attributes become effective only when the worksheet is protected.

1.7.6 Worksheet protection

1.7.6.1 sheet:protect([opts | false]) -> sheet

```
sheet:protect({
  password = "secret",
  select_locked_cells = false,
  select_unlocked_cells = true,
  format_cells = false,
  format_columns = false,
  format_rows = false,
  insert_columns = false,
  insert_rows = false,
  insert_hyperlinks = false,
  delete_columns = false,
  delete_rows = false,
  sort = false,
  auto_filter = true,
  pivot_tables = false,
  objects = false,
  scenarios = false,
})
```

Every option is boolean. `sheet:protect(false)` removes protection.

```
local protection = sheet:get_protection()
print(protection and protection.password_hash)
```

1.7.7 Workbook structure protection

1.7.7.1 workbook:protect([opts | false]) -> workbook

```
workbook:protect({
  password = "secret",
  structure = true,
```

```
    windows = false,
})
```

structure prevents ordinary sheet-structure changes. windows protects workbook windows in applications that support the option.

```
workbook:protect(false)
local protection = workbook:get_protection()
```

Classic XLSX protection is **not encryption**. The password is transformed with an old hash and is limited to 15 bytes. It primarily prevents accidental edits.

1.7.8 Manual page breaks

```
sheet:add_row_page_break(40)
sheet:add_row_page_break(80)
sheet:add_column_page_break("H")
sheet:add_column_page_break(12)
```

Rows and columns are **1-based** here, matching XLSX storage. A column can be provided as letters or as a number.

```
local rows = sheet:get_row_page_breaks()
local columns = sheet:get_column_page_breaks()
```

```
sheet:clear_page_breaks()
```

1.7.9 Document properties

1.7.9.1 workbook:set_properties(opts) -> workbook

```
workbook:set_properties({
    title = "Annual report",
    subject = "2026 results",
    creator = "Julien Freyermuth",
    description = "Report generated with lua-xlsx",
    keywords = { "report", "sales", "2026" },
    category = "Reporting",
    company = "My company",
    manager = "Department manager",
})
```

keywords accepts either a string or a dense string array. Core properties are written to docProps/core.xml; company and manager are written to docProps/app.xml.

```
local properties = workbook:get_properties()
print(properties.title, properties.creator, properties.company)
```

```
workbook:set_properties(false)
```

1.7.10 Reading 1.5 information

Objects returned by `xlsx.open()` expose:

```
local properties = workbook:get_properties()
local workbook_protection = workbook:get_protection()

local rich = sheet:get_rich_text(row, col)
local sheet_protection = sheet:get_protection()
local row_breaks = sheet:get_row_page_breaks()
local column_breaks = sheet:get_column_page_breaks()
local sparklines = sheet:get_sparklines()
local rules = sheet:get_conditional_formats()
local charts = sheet:get_charts()
```

`get_charts()` returns the type, series references, grouping, legend, data labels, supported axis settings, colors, markers, and line width.

1.7.11 Complete 1.5.0 example

```
local xlsx = require("xlsx")

local wb = xlsx.new()
wb:set_properties({
    title = "2026 dashboard",
    creator = "lua-xlsx",
    company = "My company",
})
wb:protect({ password = "structure", structure = true })

local sh = wb:add_sheet("Dashboard")
sh:write_rows({
    { "Month", "Sales", "Target", "Difference", "Trend" },
    { "January", 120, 100, 20 },
    { "February", 90, 110, -20 },
    { "March", 145, 130, 15 },
})

sh:write_rich_text(5, 0, {
    { text = "Legend: ", bold = true },
    { text = "green = target reached", font_color = "008000" },
})

sh:add_conditional_format("B2:B4", {
    type = "color_scale",
    min_color = "F8696B",
    mid_type = "percentile",
    mid_value = 50,
    mid_color = "FFEB84",
    max_color = "63BE7B",
})

sh:add_conditional_format("D2:D4", {
    type = "icon_set",
    icons = "3_traffic_lights",
    value_type = "number",
    values = { -10, 0, 10 },
})

for row = 2, 4 do
    sh:add_sparkline("E" .. row, "B" .. row .. ":D" .. row, {
        type = "line",
        color = "4472C4",
        show_high = true,
        show_low = true,
    })
end

sh:add_chart({
    type = "area",
    title = "Sales and targets",
    categories = "A2:A4",
    series = {
        { name_ref = "B1", values = "B2:B4", color = "4472C4" },
        { name_ref = "C1", values = "C2:C4", color = "ED7D31" },
    },
},
```

```

    grouping = "standard",
    legend_position = "bottom",
    x_axis = { title = "Month" },
    y_axis = { title = "Value", min = 0 },
    row = 0,
    col = 6,
})

sh:write(1, 1, 120, xlsx.style({ locked = false, fill_color = "FFF2CC" }))
sh:protect({ password = "input", select_unlocked_cells = true })
sh:add_row_page_break(40)
sh:set_print_area("A1:M40")

assert(wb:save("dashboard.xlsx"))

```

1.8 1900 and 1904 dates

```

sh:write(0, 0, xlsx.date(2026, 7, 18))
sh:write(0, 1, xlsx.datetime(2026, 7, 18, 13, 45, 30))

```

The constructors validate real calendar dates, leap years, years 1..9999, hours 0..23, minutes and seconds 0..59, and integer argument types.

Helpers:

```

local serial1900 = xlsx.date_to_serial(2026, 7, 18)
local serial1904 = xlsx.date_to_serial(2026, 7, 18, 0, 0, 0, true)

print(xlsx.serial_to_iso(serial1900, false))
print(xlsx.serial_to_iso(serial1904, false, true))

```

The writer emits the correct serial for the workbook's date_system. The reader automatically detects <workbookPr date1904="1"/>.

1.9 Reading XLSX files

1.9.1 xlsx.open(path [, opts])

```

local wb, err = xlsx.open("report.xlsx")
assert(wb, err)

```

Opening checks file size, optionally validates the archive with Babet, verifies ZIP metadata and CRCs, and parses the required XML parts.

1.9.2 Workbook methods

```

print(wb:date_system())
for _, name in ipairs(wb:sheet_names()) do print(name) end
local sheet = assert(wb:sheet("Data"))

```

sheet() also accepts a one-based sheet index.

The active sheet and defined names are available separately:

```

print(wb:get_active_sheet()) -- active sheet name

for _, item in ipairs(wb:get_defined_names()) do
    print(item.name, item.reference, item.local_sheet, item.hidden, item.comment)
end

```

```

local global = wb:get_defined_name("VATRate")
local local_name = wb:get_defined_name("LocalTotal", "Summary")

```

local_sheet is a one-based sheet index in returned tables; lookup accepts the index or sheet name.

1.9.3 Values and formulas

sheet:read(row, col) keeps the historical contract and returns the cached cell value. A formula without a cache therefore returns nil.

```

local value = sheet:read(0, 0)
local formula = sheet:get_formula(3, 1)
if formula then
    print(formula.expression, formula.cached_value, formula.formula_type)
end

```

Formula objects also expose ref and shared_index when present. A secondary shared formula may have no resolved expression; it can be inspected but cannot be written directly.

1.9.4 Styles

```

local style = sheet:get_style(0, 0)
if style then
    print(style.bold, style.font_name, style.font_size)
    print(style.number_format)
    print(style.border and style.border.bottom and style.border.bottom.style)
end

```

The returned style may be passed to write() or set_style(). Only the supported subset is exposed; themes, indexed colors, diagonal borders, and advanced variants may be omitted.

1.9.5 Layout

```

print(sheet:get_column_width(0))
print(sheet:get_row_height(0))
local rows, cols = sheet:get_frozen_panes()
print(rows, cols)
print(sheet:get_auto_filter())
print(sheet:is_row_hidden(4))
print(sheet:is_column_hidden(7))
print(sheet:get_tab_color())
print(sheet:get_visibility())

```

1.9.6 Data validation, conditional formatting and comments

```

for _, rule in ipairs(sheet:get_data_validations()) do
    print(rule.ref, rule.type, rule.operator, rule.formula1, rule.formula2)
    if rule.values then print(table.concat(rule.values, ", ")) end
end

for _, rule in ipairs(sheet:get_conditional_formats()) do
    print(rule.ref, rule.type, rule.operator, rule.formula1)
    if rule.style then print(rule.style.fill_color) end
end

local comment = sheet:get_comment(2, 0)
if comment then print(comment.author, comment.text, comment.ref) end

for _, item in ipairs(sheet:get_comments()) do
    print(item.row, item.col, item.author, item.text)
end

```


Validation operands are exposed as Excel/XML strings. Simple inline lists are also decoded into values. Conditional rules are returned in priority order. Advanced unknown rules may be inspected by their XML type but are not promised to be writable again.

1.9.7 Merged cells

```
for _, range in ipairs(sheet:merged_cells()) do print(range) end
```

1.9.8 Hyperlinks

```
local link = sheet:get_hyperlink(2, 4)
if link then print(link.target, link.internal, link.tooltip, link.ref) end
for _, item in ipairs(sheet:hyperlinks()) do print(item.ref, item.target) end
```

1.9.9 Dimensions and row iteration

```
local maxrow, maxcol = sheet:dims()
for row in sheet:rows() do
    for col = 1, row.n do print(row[col]) end
end
```

rows() iterates over cached values and preserves intermediate empty rows.

1.10 ZIP validation and limits

Option	Default	Maximum
max_file_size	256 MiB	no extra internal ceiling
max_entries	10,000	100,000
max_entry_size	64 MiB	8 GiB
max_total_size	512 MiB	64 GiB
max_path_length	4,096 bytes	1 MiB
max_total_name_bytes	16 MiB	64 MiB
max_compression_ratio	200	1,000,000,000
use_babet	true	boolean
validate_archive	true	boolean

One example per limit:

```
xlsx.open("a.xlsx", { max_file_size = 32 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_entries = 2000 })
xlsx.open("a.xlsx", { max_entry_size = 16 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_total_size = 128 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_path_length = 1024 })
xlsx.open("a.xlsx", { max_total_name_bytes = 1024 * 1024 })
xlsx.open("a.xlsx", { max_compression_ratio = 100 })
```

Combined untrusted-file profile:

```
local wb, err = xlsx.open("import.xlsx", {
    max_file_size = 64 * 1024 * 1024,
    max_entries = 5000,
    max_entry_size = 16 * 1024 * 1024,
    max_total_size = 128 * 1024 * 1024,
    max_path_length = 2048,
    max_total_name_bytes = 2 * 1024 * 1024,
    max_compression_ratio = 100,
    use_babet = true,
    validate_archive = true,
```

```
})
assert(wb, err)
```

The Lua reader rejects multi-disk ZIPs, ZIP64, encryption, unsupported methods, unsafe or duplicate names, inconsistent local headers or data descriptors, overlapping entries, size mismatches, CRC failures and truncated DEFLATE data.

1.11 DataFrame

A DataFrame contains unique, non-empty string columns and rows records.

1.11.1 Construction

1.11.1.1 DF.from_rows(matrix [, opts]) opts.header uses the first row as names. opts.columns supplies explicit names when header is false.

```
local d = DF.from_rows({
  { "name", "age" },
  { "Alice", 30 },
  { "Bob", 25 },
}, { header = true })
```

Duplicate column names are rejected.

1.11.1.2 DF.from_records(records [, columns])

```
local d = DF.from_records({
  { name = "Alice", age = 30 },
  { name = "Bob", age = 25 },
}, { "name", "age" })
```

Without columns, string keys are inferred and sorted.

1.11.1.3 DF.from_sheet(sheet [, opts])

```
local d = DF.from_sheet(assert(wb:sheet("Data")), {
  header = true,
})
```

1.11.2 Introspection

```
print(d:nrow(), d:ncol())
print(table.concat(d:colnames(), ", "))
local ages = d:column("age")
```

d:iter() yields copies:

```
for row in d:iter() do
  row.age = 0 -- does not modify d
end
```

1.11.3 Non-destructive transformations

```
local adults = d:filter(function(row) return row.age >= 18 end)
local names = d:select("name")
local renamed = d:rename({ age = "years" })
local doubled = d:mutate("double_age", function(row) return row.age * 2 end)
local asc = d:sort("age")
local desc = d:sort("age", { desc = true })
local first = d:head(10)
local last = d:tail(10)
```

filter and mutate callbacks receive a copy. filter, sort, head, tail and iter do not share row tables with the source. Rename collisions and repeated selections are rejected. Sort is stable and keeps nil last.

1.11.4 Grouping and aggregation

```
local grouped = sales:groupby("city", "product"):agg({
  total = { "sum", "amount" },
  average = { "mean", "amount" },
  minimum = { "min", "amount" },
  maximum = { "max", "amount" },
  first = { "first", "amount" },
  last = { "last", "amount" },
  count = { "count" },
})
```

Functions: sum, mean/avg, min, max, count, first, last.

Group keys support nil, booleans, binary strings and numbers. The internal key encoding is type- and length-prefixed, so embedded separators or NUL bytes cannot merge distinct groups.

```
local counts = d:groupby("city"):count()
```

1.11.5 Output

```
local records = d:to_records()
local rows = d:to_rows()
local no_header = d:to_rows({ header = false })
print(d:tostring(20))
d:show(20)
```

1.12 Error contract

Caller mistakes raise Lua errors: invalid types or options, invalid names or dates, out-of-range indices and inconsistent DataFrame operations.

Filesystem and data failures normally return nil, err:

```
local wb, err = xlsx.open("missing.xlsx")
if not wb then
  io.stderr:write(err, "\n")
end
```

workbook:sheet() follows the same convention for a corrupt or missing sheet part. Workbook:save() checks writing, flushing, closing and publication and no longer reports success after a real write failure.

1.13 Tests and interoperability

```
./run_tests.sh
```

The harness covers write/read round trips, Unicode, XML entities, CRC corruption, 1900/1904 dates, styles and number formats, dimensions, frozen panes, filters, merges and formulas, data validation, conditional formatting, comments, hidden rows and columns, sheet visibility, tab colors, active-sheet selection, global and local defined names, Excel and XML limits, Babet calls, DataFrame copy semantics, collision-free grouping and real openpyxl interoperability.

Babet is resolved in this order: BABET_BIN, the local bin/babet file, then babet from PATH. The harness creates a temporary Python virtual environment, installs openpyxl>=3.1,<4 and Pillow>=10,<12, runs the interoperability round trip separately with Babet and standard Lua when

both are available, and removes the venv, pip cache and temporary files at exit, including after a failure. A global openpyxl installation is neither used nor required.

This phase requires python3, the venv module, pip inside the venv and access to the configured Python package index. Debian and Ubuntu systems may need the python3-venv package.

```
cp /path/to/babet bin/babet
chmod +x bin/babet
./run_tests.sh
```

```
BABET_BIN=../babet/bin/babet ./run_tests.sh
```

```
LUA_BIN=/usr/bin/lua5.5 ./run_tests.sh
```

```
OPENPYXL_SPEC='openpyxl==3.1.5' PILLOW_SPEC='pillow==11.3.0' ./run_tests.sh
```

1.14 Building the PDFs

```
cd doc
./build_doc.sh
```

Outputs:

- documentation-fr.pdf;
- documentation-en.pdf.

Pandoc and XeLaTeX or LuaLaTeX are required. Auxiliary LaTeX files are kept in a temporary directory.

1.15 Known limitations

- Writing uses STORED ZIP entries; reading supports STORED and DEFLATE.
- The internal Lua parser does not support ZIP64.
- No pivot tables, XLSM macros, combined or 3D charts, secondary axis, SVG images, or advanced multi-area page layouts.
- Sparklines use the OOXML x14 extension. Excel and LibreOffice preserve it, while openpyxl 3.1.5 removes it when saving the workbook again.
- Classic XLSX protection does not encrypt the workbook and limits passwords to 15 bytes.
- Comments are read and written as plain text; rich-text runs, custom dimensions, and custom positions are not preserved.
- Formulas are read and written but not calculated. Secondary shared formulas without a resolved expression are not written back.
- Style reading covers the supported subset; themes, indexed colors, diagonal borders, and advanced options may be omitted.
- No streaming reader: the file and selected XML parts remain in memory.
- The XML scanner is XLSX-specific, not a general XML parser.
- No Unicode normalization or complete Unicode case folding for sheet names; case-insensitive duplicate detection is reliable for ASCII.